Java Object-Oriented Programming & Low-Level Design (LLD)

The Comprehensive Industry Mastery Guide (Java Edition): From JVM Architecture, Memory Layout, and Bytecode Dispatch to SOLID, GoF Design Patterns, and Enterprise Machine Coding

Chapter 1: The Paradigm Shift & Java Memory Architecture

A rigorous exploration of programming paradigms, JVM runtime internals (Heap, Stack, Metaspace), why Java is strictly pass-by-value, and the famous Integer Cache trap.

1.1 The Evolution of Object-Oriented Programming in Java

Before Java emerged in 1995, procedural languages like C and early C++ dominated industry computing. However, procedural systems suffered from architectural entropy: data structures and global state were decoupled from functions, leading to fragile spaghetti logic where altering a struct field would cascade into hundreds of broken functions. Java was engineered from the ground up to enforce strict Object-Oriented principles, platform independence ('Write Once, Run Anywhere'), and automated memory management.

Dimension	Procedural (C)	Object-Oriented (Java)	Functional (Haskell / Streams)
Core Unit	Procedures / Functions	Classes & Objects	Pure Mathematical Functions
State Management	Decoupled global / local structs	Encapsulated private fields within objects	Immutable state & transformation pipelines
Memory Management	Manual (`malloc` / `free`) - error prone	Automatic Garbage Collector (JVM)	Garbage Collected / Immutable allocations
Extensibility	Conditional switch-cases & function pointers	Polymorphism, Interfaces & Subtyping	Higher-order functions & Currying
Concurrency Model	Manual OS threads & raw mutexes	Java Memory Model (`volatile`, `synchronized`)	Lock-free immutable parallelism

1.2 The Four Pillars of OOP in Java

Java enforces the four core OOP pillars as fundamental language semantics:

- **1. Encapsulation:** Bundling data (fields) and methods operating on that data into a cohesive class, while restricting direct access using access modifiers (`private`, `protected`, `public`).
- **2. Abstraction:** Hiding internal implementation complexity and exposing a clean public contract using `interface` and `abstract class` declarations.
- **3. Inheritance:** Acquiring properties and behaviors from a superclass via the `extends` keyword to model 'Is-A' taxonomies and enable code reuse.
- **4. Polymorphism:** Enabling a single reference type or method signature to dynamically execute different runtime behaviors via dynamic method dispatch.

1.3 Inside the Java Virtual Machine (JVM) Architecture

To write production-grade Java, an engineer must understand the physical memory regions managed by the JVM during execution:

- **1. JVM Stack (Thread-Private):** Each operating thread possesses its own stack. Whenever a method is invoked, a new Stack Frame is pushed. A stack frame contains: (1) Local Variable Array, (2) Operand Stack, and (3) Frame Data (runtime constant pool references). Deallocated automatically when the method returns (LIFO).
- **2. JVM Heap (Thread-Shared):** The shared memory zone where ALL Java objects and arrays reside. Divided into Young Generation (Eden space, Survivor S0/S1 spaces) and Old/Tenured Generation. Managed by the Garbage Collector.
- **3. Metaspace / Method Area (Thread-Shared):** Stores class-level metadata, runtime constant pool, field and method bytecodes, and static variables. In Java 8+, Metaspace replaced the old PermGen and is allocated out of native OS memory.
- **4. Program Counter (PC) Register:** Holds the address of the current JVM bytecode instruction being executed by each thread.

Memory Region	Lifetime	Thread Scope	Contents	Exception on Overflow
JVM Stack	Method invocation scope	Thread-Private	Primitives, Object reference pointers, Stack frames	`StackOverflowError`
JVM Heap	Application runtime (until GC)	Thread-Shared	All instantiated Objects, Arrays, Instance fields	`OutOfMemoryError: Java heap space`
Metaspace	ClassLoader lifecycle	Thread-Shared	Class definitions, bytecode, Static variables, Constant Pool	`OutOfMemoryError: Metaspace`

1.4 Why Java is Strictly 100% Pass-by-Value (The Ultimate Clarification)

One of the most persistent misconceptions in Java interviews is that 'primitives are passed by value, but objects are passed by reference.' This is FALSE. Java is strictly 100% PASS-BY-VALUE in all scenarios.

When an object variable is passed to a method, you are NOT passing the object itself, nor are you passing a C++ style reference alias. You are passing a COPY OF THE MEMORY ADDRESS (reference) stored on the caller's stack frame. Both the caller and the callee hold independent copies of the reference pointing to the same heap object.

```
// --- Proof of Java Strict Pass-by-Value Semantics (JAVA) ---
public class ParameterPassingMechanics {
    static class User {
        String name;
        User(String name) { this.name = name; }
    }

    // 1. Primitive: Pass-by-value of raw bits
    public static void modifyPrimitive(int x) {
        x = 999; // Alters only the local stack frame copy!
    }

    // 2. Object Mutation: Operates on the shared heap instance
    public static void mutateObject(User u) {
        u.name = "Alice"; // Mutates the object on the heap!
    }

    // 3. Object Reassignment: Proves Java is Pass-by-Value!
```

```
    public static void reassignReference(User u) {
        // 'u' is a local copy of the reference on the stack.
        // Pointing 'u' to a new object does NOT alter the caller's reference!
        u = new User("Bob");
    }

    public static void main(String[] args) {
        int num = 10;
        modifyPrimitive(num);
        System.out.println("num: " + num); // Output: 10 (Unchanged!)

        User user = new User("Initial");
        mutateObject(user);
        System.out.println("After mutate: " + user.name); // Output: Alice (Field mutated!)

        reassignReference(user);
        System.out.println("After reassign: " + user.name); // Still Alice! NOT Bob!
    }
}
```

1.5 Autoboxing, Wrapper Classes & The Integer Cache Trap

Java provides Object Wrapper Classes (`Integer`, `Long`, `Double`, `Boolean`) for every primitive type. Autoboxing is the automatic conversion made by the Java compiler between primitive types and their corresponding wrapper objects (e.g., `int` to `Integer`).

However, wrapper objects are heap instances compared via memory address when using `==`. To conserve memory, the Java Language Specification mandates caching for `Integer` objects between -128 and 127 via `Integer.valueOf()`.

```
// --- The Java Integer Cache Trap (-128 to 127) (JAVA) ---
public class IntegerCacheTrap {
    public static void main(String[] args) {
        // Within Cache Range (-128 to 127): Reuses cached instances!
        Integer a = 127;
        Integer b = 127;
        System.out.println(a == b);      // TRUE! (Both point to identical cached heap object)
        System.out.println(a.equals(b)); // TRUE! (Values match)

        // Outside Cache Range: Instantiates new distinct heap objects!
        Integer x = 128;
        Integer y = 128;
        System.out.println(x == y);      // FALSE! (Different heap memory addresses!)
        System.out.println(x.equals(y)); // TRUE! (Logical values match)
    }
}
```

★ Senior Software Engineer Rule of Equality

1. ALWAYS compare Java Objects using `.equals()` rather than `==`.
2. Using `==` on Wrapper Classes causes catastrophic, intermittent bugs when values cross the 127 cache boundary.
3. Use `==` strictly for primitive type comparison (`int`, `boolean`, `double`) or checking `null`.

Chapter 2: Classes, Objects & The Java Type System

Deep exploration of class structure, execution ordering of initializers, 'this' keyword mechanics, HotSpot JVM in-memory object layout, and the sacred equals() / hashCode() contract.

2.1 Class Blueprint vs. Heap Object Instance

In Java, a Class is a compile-time schema loaded into the Metaspace. An Object is a dynamically allocated chunk of contiguous memory on the JVM Heap created via the `new` keyword or reflection. Every class in Java implicitly or explicitly extends `java.lang.Object`, the ultimate root of the Java class hierarchy.

2.2 Anatomy of a Class: Fields, Methods & Initializer Blocks

A Java class comprises four structural elements: (1) Fields, (2) Methods, (3) Constructors, and (4) Initializer Blocks. The exact execution order during class loading and instantiation is a classic interview question:

- **1. Static Initializers (`static {}`):** Execute exactly ONCE when the class is first loaded into memory by the ClassLoader. Runs top-to-bottom.
- **2. Instance Initializers (`{}`):** Execute EVERY time a new instance is created, running immediately BEFORE the constructor body.
- **3. Constructor (`ClassName(...)`):** Executes after instance variable assignments and instance initializer blocks have finished.

```
// --- Exact Initialization Order in Java (JAVA) ---
public class InitializationOrderDemo {
    // 1. Static field & static block (Executed once on class load)
    static int staticCounter = initStatic();
    static { System.out.println("2. Static Block Executed"); }

    // 2. Instance field & instance block (Executed on every 'new')
    int instanceVal = initInstance();
    { System.out.println("4. Instance Initializer Block Executed"); }

    // 3. Constructor
    public InitializationOrderDemo() {
        System.out.println("5. Constructor Body Executed");
    }

    private static int initStatic() {
        System.out.println("1. Static Field Initialized");
        return 100;
    }

    private int initInstance() {
        System.out.println("3. Instance Field Initialized");
        return 200;
    }

    public static void main(String[] args) {
        System.out.println("--- Instantiating Object 1 ---");
        new InitializationOrderDemo();
        System.out.println("--- Instantiating Object 2 ---");
        new InitializationOrderDemo();
    }
}
```

2.3 The Four Powers of the 'this' Keyword

The `this` keyword is an implicit reference variable that points to the current object instance whose method or constructor is being invoked. It serves four distinct architectural functions:

- **1. Variable Shadowing Resolution:** Disambiguates between instance fields and local parameters of the same name: `this.name = name;`.
- **2. Explicit Constructor Chaining:** Invoking an overloaded constructor from within another constructor: `this(arg1, arg2);`. Must be the very FIRST statement in the constructor!
- **3. Passing Current Instance as Parameter:** Passing the active object to an external service or event listener: `eventPublisher.register(this);`.
- **4. Fluent Method Chaining (Method Chaining / Builder):** Returning `this` at the end of mutating methods to enable fluent API chaining: `builder.setA().setB();`.

2.4 In-Memory Layout of a Java Object (HotSpot JVM)

What does an object actually look like in physical RAM? On a 64-bit HotSpot JVM, every object consists of three distinct segments:

Component	Byte Size (64-bit with Compressed OOPs)	Internal Contents
Mark Word (Header)	8 Bytes (64 bits)	Identity HashCode, GC Age bits (4 bits for 15 promotions), Lock state flags, Biased Locking info
Klass Pointer (Header)	4 Bytes (Compressed OOPs) or 8 Bytes	Pointer to class metadata residing in Metaspace (identifies runtime type)
Array Length (Header)	4 Bytes (Arrays ONLY)	Integer denoting the number of elements in the array
Instance Data	Variable (aligned to 4/8 bytes)	Actual primitive values and 4-byte reference pointers to other heap objects
Padding	0 to 7 Bytes	Zero-filled bytes added to pad the total object size to a multiple of 8 bytes (64-bit alignment)

2.5 Object Identity (==) vs. Logical Equality (.equals())

By default, the `.equals()` method inherited from `java.lang.Object` simply executes `this == obj` (memory address identity). If you want two distinct objects with identical fields to be considered equal, you MUST override `.equals()`.

2.6 The Sacred equals() and hashCode() Contract

Violating the contract between `equals()` and `hashCode()` is one of the most common causes of silent, catastrophic bugs in production Java applications. The Java Language Specification mandates:

- **Rule 1 (Consistency):** If `o1.equals(o2)` is `true`, then `o1.hashCode()` MUST EQUAL `o2.hashCode()`.
- **Rule 2 (Collisions):** If `o1.hashCode() == o2.hashCode()`, `o1.equals(o2)` is NOT necessarily true (hash collision).
- **Consequence of Violation:** If you override `equals()` without overriding `hashCode()`, hash-based collections (`HashMap`, `HashSet`, `Hashtable`) will store equal objects in completely different hash buckets, making it impossible to retrieve them via `.get()`!

```
// --- Robust equals() and hashCode() Contract Implementation (JAVA) ---
import java.util.Objects;
```

```
import java.util.HashMap;
import java.util.Map;

public class Employee {
    private final int id;
    private final String department;

    public Employee(int id, String dept) {
        this.id = id;
        this.department = dept;
    }

    // 1. Standard Robust equals() Implementation
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // Step 1: Identity check
        if (o == null || getClass() != o.getClass()) return false; // Step 2: Exact type check
        Employee employee = (Employee) o;                    // Step 3: Safe cast
        return id == employee.id && Objects.equals(department, employee.department);
    }

    // 2. Mandatory Corresponding hashCode()
    @Override
    public int hashCode() {
        return Objects.hash(id, department); // Combines field hashes
    }

    public static void main(String[] args) {
        Map<Employee, String> employeeMap = new HashMap<>();
        Employee e1 = new Employee(101, "Engineering");
        employeeMap.put(e1, "Tech Lead");

        // Lookup with an equal, distinct object instance:
        Employee lookupKey = new Employee(101, "Engineering");
        System.out.println("Role: " + employeeMap.get(lookupKey));
        // Output: "Tech Lead" (Works because both equals() and hashCode() are overridden!)
    }
}
```

★ Why Cloneable is Flawed (Joshua Bloch's Recommendation)

The `Cloneable` interface in Java is widely recognized as a broken design: it has no methods, and `Object.clone()` is protected and bypasses constructors, performing shallow bitwise copies.

Best Practice: Avoid `Cloneable`. Instead, implement a Copy Constructor (`public Employee(Employee other)`) or a static factory method (`Employee.copy(other)`). This provides type-safe, explicit deep copying without reflection.

Chapter 3: Object Lifecycle, Garbage Collection & Resource Management

Mastering the stages of a Java object, constructor chaining, the deprecation of finalize(), modern try-with-resources, Generational GC mechanics, and Java's 4 Reference Types.

3.1 The Life Cycle of a Java Object

A Java object moves through a deterministic series of JVM phases: (1) Class Loading & Verification, (2) Heap Memory Allocation, (3) Default Zero-Value Field Initialization (e.g., `0`, `false`, `null`), (4) Explicit Field Initialization

& Instance Block `{}` Execution, (5) Constructor Execution via `super()`, (6) Active Lifetime, (7) Unreachability Isolation, and (8) Garbage Collection Reclamation.

3.2 Constructor Chaining & Super Rules

When an object is created, its parent's constructor MUST be invoked before the child constructor executes. The compiler enforces strict syntactic rules:

- **1. Mandatory First Statement:** The call to `super(...)` or `this(...)` must be the absolute FIRST line in any constructor. You cannot place code before it.
- **2. Mutual Exclusivity:** A constructor can contain `super()` OR `this()`, but NEVER both simultaneously.
- **3. Implicit Default super():** If you do not explicitly invoke `super()` or `this()`, the Java compiler automatically inserts an invisible `super();` call to the parent's no-argument constructor. If the parent does not have a no-arg constructor, a compilation error occurs.

```
// --- Java Constructor Chaining (super and this) (JAVA) ---
class Vehicle {
    protected String vin;
    public Vehicle(String vin) {
        this.vin = vin;
        System.out.println("1. Vehicle Constructor: VIN " + vin);
    }
}

class ElectricCar extends Vehicle {
    private int batteryCapacity;

    public ElectricCar(String vin) {
        this(vin, 75); // Constructor Chaining: Delegates to overloaded constructor
    }

    public ElectricCar(String vin, int batteryCapacity) {
        super(vin); // Parent constructor MUST be first!
        this.batteryCapacity = batteryCapacity;
        System.out.println("2. ElectricCar Constructor: " + batteryCapacity + " kWh");
    }
}
```

3.3 Why finalize() is Deprecated and Hazardous

Historically, Java provided `protected void finalize()` in `Object`. Java 9 officially deprecated `finalize()`, and modern Java marks it for complete removal. Finalizers are dangerous because: (1) They run on an arbitrary, low-priority JVM thread with no execution time guarantee; (2) They can accidentally resurrect dead objects by assigning `this` to a static field; (3) Objects with finalizers must survive at least one extra GC cycle, causing severe memory bloat and latency spikes.

3.4 Deterministic Resource Cleanup: AutoCloseable & try-with-resources

Introduced in Java 7, `try-with-resources` provides deterministic cleanup for OS resources (sockets, database connections, file handles). Any class that implements `java.lang.AutoCloseable` will have its `close()` method invoked automatically when the `try` block exits, even if an unhandled exception is thrown.

```
// --- Deterministic Cleanup with try-with-resources (JAVA) ---
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
```

```
public class ResourceManagementDemo {
    // Custom AutoCloseable Resource
    static class DatabaseConnection implements AutoCloseable {
        public void executeQuery(String sql) { System.out.println("Executing: " + sql); }
        @Override
        public void close() { System.out.println("DatabaseConnection cleanly closed."); }
    }

    public static void main(String[] args) {
        // try-with-resources guarantees deterministic closure!
        try (DatabaseConnection conn = new DatabaseConnection()) {
            conn.executeQuery("SELECT * FROM users");
            // If an exception occurs here, conn.close() is STILL guaranteed to execute!
        } // 'conn.close()' runs automatically at this exact boundary
    }
}
```

3.5 Garbage Collection Mechanics: The Generational Hypothesis

The JVM Garbage Collector relies on the Weak Generational Hypothesis: empirically, over 95% of allocated objects die shortly after creation (temporary strings, local loop variables, DTOs). To optimize performance, the JVM divides the Heap into distinct generations:

Heap Generation	Sub-Regions	Allocation & Collection Characteristics
Young Generation	Eden Space, Survivor 0 (From), Survivor 1 (To)	All new objects allocated in Eden. Minor GC runs frequently using fast Copying algorithms. Survivors age across S0 and S1.
Old / Tenured Generation	Contiguous Tenured Space	Objects that survive 15 Minor GC cycles (configurable via `-XX:MaxTenuringThreshold`) are promoted here. Collected via Major / Full GC.
Metaspace (Native OS)	Klass Metadata, Constant Pool	Allocated from native RAM. Reclaimed only when entire ClassLoaders become unreferenced and are unloaded.

3.6 The Four Java Reference Types (Crucial for High-Performance Systems)

Java provides four levels of object reachability in `java.lang.ref` to allow fine-grained control over caching and memory reclamation:

- **1. StrongReference (Default):** Standard variable assignment (`User u = new User();`). As long as a strong GC root reaches the object, it will NEVER be garbage collected. If heap runs out, the JVM throws `OutOfMemoryError` rather than collecting it.
- **2. SoftReference<T>:** Objects wrapped in `SoftReference` are retained as long as memory is sufficient. The GC reclaims them ONLY as a last resort before throwing `OutOfMemoryError`. Ideal for memory-sensitive in-memory caches.
- **3. WeakReference<T>:** Does not prevent GC collection. On the very next GC run, any object referenced ONLY by weak references is reclaimed immediately. Critical for canonical mappings (`WeakHashMap`) and event listener registries to prevent memory leaks.

- **4. PhantomReference<T>:** Enqueued in a `ReferenceQueue` AFTER the object has been physically finalized and reclaimed. Used for low-level off-heap memory cleanup and post-mortem resource monitoring.

```
// --- Java WeakReference Behavior & Garbage Collection (JAVA) ---
import java.lang.ref.WeakReference;

public class WeakReferenceDemo {
    public static void main(String[] args) {
        String data = new String("Heavy Payload Cache");
        WeakReference<String> weakRef = new WeakReference<>(data);

        System.out.println("Before GC: " + weakRef.get()); // Accessible

        data = null; // Clear the only STRONG reference!
        System.gc(); // Request JVM Garbage Collection

        // On next GC, the weakly referenced object is reclaimed!
        System.out.println("After GC: " + weakRef.get()); // Prints: null
    }
}
```

> **★ Architectural Takeaway: Memory Leaks in Java**
> Java does have memory leaks! A memory leak in Java occurs when unused objects remain reachable from a GC Root (such as uncleaned static collections, unremoved event listeners, or thread pools holding ThreadLocals).
> Always clean up static collections and use `WeakReference` or `WeakHashMap` when registering observers and listeners.

Chapter 4: Encapsulation, Defensive Copying & Immutability

Deep exploration of information hiding, Java access modifier boundaries, anemic vs. rich domain models, defensive copying against state leaks, and Java 16+ records.

4.1 Information Hiding: The Real Meaning of Encapsulation

Encapsulation is not merely making fields `private` and generating getters and setters. True encapsulation is Information Hiding: enclosing data with the business behavior that governs it, protecting invariants, and shielding internal representations from external consumers so that internal changes do not break client code.

Modifier	Same Class	Same Package	Subclass (Different Package)	World (Anywhere)
`private`	YES	NO	NO	NO
`default` (no modifier)	YES	YES	NO	NO
`protected`	YES	YES	YES (Inheritance access)	NO
`public`	YES	YES	YES	YES

4.2 The Getter/Setter Anti-Pattern & Anemic Domain Models

Mindlessly exposing getters and setters for all private fields produces an Anemic Domain Model. When business logic is pulled out of entities and scattered across service classes, objects are reduced to passive data holders, causing massive code duplication and broken invariants.

```
// --- Anemic Domain Model vs Rich Encapsulated Domain Model (JAVA) ---
```

```
// BAD: Anemic Domain Model (Zero Encapsulation)
public class AnemicBankAccount {
    private double balance;
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
}
// Client code must remember to validate logic:
// if (acc.getBalance() >= amt) acc.setBalance(acc.getBalance() - amt);

// ------------------------------------------------------------
// GOOD: Rich Domain Model (Encapsulated Invariants)
// ------------------------------------------------------------
public class RichBankAccount {
    private double balance;

    public RichBankAccount(double initialDeposit) {
        if (initialDeposit < 0) throw new IllegalArgumentException("Initial deposit cannot be
negative");
        this.balance = initialDeposit;
    }

    // Business Operation: Protects invariants internally
    public void withdraw(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Withdrawal must be positive");
        if (amount > this.balance) throw new IllegalStateException("Insufficient funds");
        this.balance -= amount;
    }

    public double getBalance() { return this.balance; } // Read-only view
}
```

4.3 Defensive Copying: Preventing Sneaky State Corruption

Even when all fields are marked `private`, if a class accepts or returns a reference to a mutable object (such as `java.util.Date`, arrays, or `List`), external code can mutate the object's internal state directly from the outside! Defensive Copying creates an independent clone during both input (constructor) and output (getter).

```
// --- Defensive Copying in Java Constructors and Getters (JAVA) ---
import java.util.Date;
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;

public final class UserSession {
    private final String userId;
    private final Date loginTime;               // Mutable object!
    private final List<String> permissions;    // Mutable collection!

    // Constructor with Defensive Copying
    public UserSession(String userId, Date loginTime, List<String> permissions) {
        this.userId = userId;
        // DEFENSIVE COPY: Do NOT store the caller's Date reference directly!
        this.loginTime = new Date(loginTime.getTime());
        // DEFENSIVE COPY: Copy collection elements into a new internal list
        this.permissions = new ArrayList<>(permissions);
    }

    // Getter with Defensive Copying
    public Date getLoginTime() {
        return new Date(this.loginTime.getTime()); // Return copy! Never return raw field!
    }
```

```
    // Returning an Unmodifiable View
    public List<String> getPermissions() {
        return Collections.unmodifiableList(this.permissions); // Prevents mutation attempts!
    }
}
```

4.4 The 5 Rules for Immutable Classes & Java 16+ Records

Immutable objects are thread-safe, cannot be corrupted, and make perfect Map keys and Set elements. Joshua Bloch specifies 5 rules for immutability:

- **1. No Mutators:** Do not provide any methods that modify the object's state (no setters).
- **2. Prevent Subclassing:** Ensure the class cannot be extended (mark class `final` or use private constructors).
- **3. Make All Fields final:** Enforces single assignment and ensures thread-safe publication.
- **4. Make All Fields private:** Prevents direct client access.
- **5. Defensive Copying:** Ensure exclusive access to any mutable components.

In Java 14+ (standardized in Java 16), the `record` keyword provides first-class language support for immutable data carriers with concise syntax:

```
// --- Modern Java 16+ Immutable Records (JAVA) ---
// Java 16+ Record: Automatically generates final fields, constructor,
// getters (x() and y()), equals(), hashCode(), and toString()!
public record Point(int x, int y) {
    // Compact Constructor for Invariant Validation
    public Point {
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("Coordinates must be non-negative");
        }
    }
}

// Usage:
Point p1 = new Point(10, 20);
System.out.println(p1.x()); // 10 (Accessors do not use 'get' prefix)
System.out.println(p1);     // Point[x=10, y=20]
```

> **★ The 'Tell, Don't Ask' Principle**
>
> Instead of querying an object about its internal state, making a decision outside, and then mutating the object, tell the object directly what operation to perform.
>
> Violating Example: `if (cart.getItems().size() > 10) cart.setDiscount(0.1);`
>
> Refactored Clean Code: `cart.applyBulkDiscount();` (Encapsulates business rules inside Cart).

Chapter 5: Abstraction & Modern Java Interfaces

Mastering abstraction, abstract classes, interface contracts, Java 8+ default and static methods, resolving interface diamond conflicts, and Functional Interfaces.

5.1 What is Abstraction?

Abstraction is the design practice of separating the 'What' (the public contract) from the 'How' (the private implementation). When a client interacts with a Java `List`, it relies on methods like `.add()` and `.get()` without

caring whether the underlying memory structure is a contiguous array (`ArrayList`) or a doubly linked list (`LinkedList`). Abstraction decouples consumers from internal volatility.

5.2 Abstract Classes in Java

An Abstract Class (`abstract class`) serves as a partial template. Key characteristics:

- **1. Cannot Be Instantiated Directly:** Attempting `new MyAbstractClass()` causes a compile-time error.
- **2. Can Hold State:** Can declare instance variables, constructors, static blocks, and full access modifiers (`private`, `protected`).
- **3. Mixes Abstract and Concrete Behavior:** Can provide complete default algorithms while deferring specific steps to subclasses via `abstract` methods.

```
// --- Java Abstract Class with Template Method Pattern (JAVA) ---
// Java Abstract Class with Shared Template Method
public abstract class PaymentProcessor {
    protected String merchantId; // Abstract classes CAN hold state!

    public PaymentProcessor(String merchantId) {
        this.merchantId = merchantId;
    }

    // Concrete Template Method: Defines the execution workflow
    public final void processTransaction(double amount) {
        authenticate();
        executePayment(amount); // Polymorphic hook!
        auditLog(amount);
    }

    // Pure Abstract Method: Subclasses MUST implement!
    protected abstract boolean executePayment(double amount);

    private void authenticate() {
        System.out.println("Authenticating merchant: " + merchantId);
    }

    private void auditLog(double amount) {
        System.out.println("Transaction audited: $" + amount);
    }
}
```

5.3 Interfaces: Pure Architectural Contracts

An Interface in Java defines a pure contract specifying 'what' a class must do, but not 'how'. A class can implement multiple interfaces (`implements A, B, C`), overcoming Java's prohibition of multiple class inheritance.

Comparison Criteria	Abstract Class (`abstract class`)	Interface (`interface`)
Core Relationship	Inheritance ('What it is' - Is-A)	Contractual Capability ('What it can do' - Can-Do)
Multiple Inheritance	Single inheritance only (`extends SuperClass`)	Multiple implementation allowed (`implements A, B`)
State & Variables	Can declare instance fields of any access level	Cannot have instance state; fields are implicitly `public static final` constants
Constructors	Can declare constructors (called via `super()`)	CANNOT have constructors
Method Visibility	Can have `private`, `protected`, and	Methods are implicitly `public` (or

	`public` methods	`private` since Java 9)
Dispatch Overhead	HotSpot JVM `vtable` lookup (fast)	HotSpot JVM `itable` lookup (slightly more indirection)
Architectural Intent	Shared code and tight identity hierarchy	Loose coupling and polymorphic plug-and-play capability

5.4 Java 8+ Interface Evolution: Default & Static Methods

Historically, adding a new method to an existing interface broke every class in the world implementing that interface. To permit interface evolution (such as adding `.stream()` to `java.util.Collection`), Java 8 introduced `default` methods.

5.5 Resolving the Diamond Conflict in Interface Default Methods

If a class implements two interfaces that provide identical default method signatures, the compiler halts with an ambiguity error. The implementing class MUST explicitly override the method and choose which interface implementation to call using `InterfaceName.super.method()`:

```
// --- Resolving Interface Default Method Collision in Java (JAVA) ---
interface EngineService {
    default void start() {
        System.out.println("Engine service initialized.");
    }
}

interface ElectricalService {
    default void start() {
        System.out.println("Electrical service powered on.");
    }
}

// Resolving Ambiguity via explicit override
public class HybridCar implements EngineService, ElectricalService {
    @Override
    public void start() {
        // Disambiguate using InterfaceName.super:
        EngineService.super.start();
        ElectricalService.super.start();
        System.out.println("HybridCar is fully operational.");
    }
}
```

5.6 Functional Interfaces & Single Abstract Method (SAM)

A Functional Interface is an interface that declares exactly ONE abstract method. Marked with the `@FunctionalInterface` annotation, it serves as the foundational bridge between Object-Oriented Programming and Functional Programming in Java, enabling Lambda Expressions and Method References.

```
// --- Functional Interface & Lambda Expressions in Java (JAVA) ---
@FunctionalInterface
public interface Validator<T> {
    boolean validate(T item); // The Single Abstract Method (SAM)

    // Can still contain default and static methods:
    default Validator<T> and(Validator<T> other) {
        return item -> this.validate(item) && other.validate(item);
```

```
    }
}

// Lambda usage:
Validator<String> emailValidator = email -> email != null && email.contains("@");
```

> **★ Design Rule: Abstract Class vs. Interface**
> Use an Abstract Class when classes share significant code, internal private state, and belong to a strict natural family tree.
> Use an Interface when defining a polymorphic capability (e.g., `Serializable`, `Comparable`, `AutoCloseable`) across completely unrelated classes.

Chapter 6: Inheritance & Class Hierarchies in Java

Deep exploration of class inheritance, why Java bans multiple class inheritance, constructor sequencing, the 4 rules of method overriding, the final keyword, and the Fragile Base Class problem.

6.1 The 'Is-A' Taxonomy

Inheritance allows a subclass to acquire methods and fields from a superclass using the `extends` keyword, establishing an 'Is-A' relationship. Java supports Single Inheritance, Multilevel Inheritance, and Hierarchical Inheritance. However, Java STRICTLY PROHIBITS Multiple Class Inheritance.

6.2 Why Java Banned Multiple Class Inheritance (The Diamond Problem)

If class `D` were allowed to inherit from both `B` and `C`, and both `B` and `C` inherited from `A`, severe architectural ambiguities arise: (1) If class `A` has an instance field `int count;`, does `D` contain one copy or two copies of `count` in its heap layout? (2) If both `B` and `C` override `void execute()`, which implementation does `D` execute? To maintain simplicity, prevent JVM memory layout bloat, and eliminate constructor ambiguity, Java designers restricted classes to single inheritance while allowing multiple interface implementation.

6.3 The Four Non-Negotiable Rules of Method Overriding in Java

When a subclass provides a specific implementation of a method defined in its superclass, the JVM enforces four strict compile-time rules:

- **Rule 1 (Signature Identity):** The method name and argument parameter list must match the parent method exactly.
- **Rule 2 (Access Level Cannot Be Reduced):** The overriding method cannot be more restrictive than the parent method. If the superclass method is `protected`, the subclass method may be `protected` or `public`, but NEVER `default` or `private`.
- **Rule 3 (Covariant Return Types):** The return type of the overriding method must be identical to, or a subtype of, the return type declared in the superclass method.
- **Rule 4 (Exception Restriction):** The overriding method CANNOT throw new or broader CHECKED exceptions than declared by the superclass method. It may throw fewer exceptions, narrower subclasses of exceptions, or no exceptions at all (unchecked RuntimeExceptions are unrestricted).

```
// --- The 4 Rules of Method Overriding in Java (JAVA) ---
import java.io.IOException;
import java.io.FileNotFoundException;

class StorageService {
```

```
    // Parent method: protected, returns Object, throws IOException
    protected Object fetchFile(String path) throws IOException {
        System.out.println("Fetching raw object from: " + path);
        return new Object();
    }
}

class S3StorageService extends StorageService {
    // 1. Access widened from 'protected' to 'public' (LEGAL)
    // 2. Return type narrowed from Object to String (Covariant Return - LEGAL)
    // 3. Exception narrowed from IOException to FileNotFoundException (LEGAL)
    @Override
    public String fetchFile(String path) throws FileNotFoundException {
        System.out.println("Fetching S3 string from: " + path);
        return "S3_BUCKET_PAYLOAD";
    }
}
```

6.4 The 'final' Keyword in Java

The `final` modifier establishes immutability and restriction across three dimensions:

Dimension	Syntax	Architectural Effect
Final Variable	`final int MAX = 100;`	Constant value. Once assigned, cannot be reassigned. For object references, the memory address is fixed (internal fields can still mutate unless object is immutable).
Final Method	`public final void verify();`	Cannot be overridden by any subclass. Enables HotSpot JVM JIT compiler to inline the method bytecode for high performance.
Final Class	`public final class String {}`	Cannot be extended/subclassed. Guarantees security and immutable integrity (e.g., `String`, `Integer`).

6.5 Favor Composition Over Inheritance (The Fragile Base Class Problem)

Inheritance exposes the internal implementation details of superclasses to subclasses (white-box reuse). Changes in a superclass can silently break subclasses without compiler warnings. Joshua Bloch famously demonstrated this by extending `HashSet` to count added elements:

```
// --- The Fragile Base Class Problem with Java Inheritance (JAVA) ---
// BROKEN: Fragile Base Class Trap
import java.util.HashSet;
import java.util.Collection;
import java.util.List;

public class InstrumentedHashSet<E> extends HashSet<E> {
    private int addCount = 0;

    @Override
    public boolean add(E e) {
        addCount++;
        return super.add(e);
    }
```

```
    @Override
    public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return super.addAll(c); // DANGER: HashSet.addAll() internally calls add()!
    }

    public int getAddCount() { return addCount; }

    public static void main(String[] args) {
        InstrumentedHashSet<String> set = new InstrumentedHashSet<>();
        set.addAll(List.of("A", "B", "C"));
        // We added 3 elements. What is addCount?
        System.out.println("Add count: " + set.getAddCount());
        // OUTPUT: 6! (Double counted because super.addAll calls our overridden add()!)
    }
}
```

★ Solution: Composition with Forwarding (Decorator Pattern)

Instead of extending a concrete class (`extends HashSet`), give your class a private reference to the interface (`private final Set<E> set;`).

This completely decouples your class from internal superclass implementation quirks, creating resilient, testable code.

Chapter 7: Polymorphism & JVM Dynamic Dispatch Internals

Mastering compile-time vs. runtime polymorphism, method overloading precedence, HotSpot JVM bytecode instructions (invokevirtual, invokespecial), vtable/itable internals, and Java 16+ pattern matching.

7.1 Compile-Time vs. Run-Time Polymorphism

Polymorphism allows objects of different types to be treated uniformly through a shared interface, while executing behavior tailored to their actual runtime class.

Dimension	Compile-Time Polymorphism (Static)	Run-Time Polymorphism (Dynamic)
Resolution Time	Compile time by `javac` compiler	Runtime by JVM HotSpot Execution Engine
Mechanism	Method Overloading	Method Overriding (Dynamic Method Dispatch)
Bytecode Instruction	Statically bound via exact parameter types in constant pool	`invokevirtual` or `invokeinterface` via `vtable`/`itable`
Execution Overhead	Zero runtime overhead; direct branch	Minimal overhead: Table lookup in class metadata

7.2 Method Overloading & The Compiler Precedence Rules

Method Overloading allows multiple methods to share the same name within the same class, differing only in parameter count, types, or order. Overloading CANNOT be resolved by return type alone.

When an overloaded method is called with ambiguous arguments, the Java compiler selects the method following a strict 4-phase precedence hierarchy:

- **Phase 1: Exact Match / Widening Primitive Conversion:** (e.g., `int` widens to `long`, then to `double`).

- **Phase 2: Autoboxing Conversion:** (e.g., `int` autoboxes to `Integer`).
- **Phase 3: Variable-Arity (Varargs):** (e.g., `int... args` is evaluated only as a last resort).

```
// --- Java Method Overload Resolution Precedence (JAVA) ---
public class OverloadResolution {
    static void print(long a)      { System.out.println("1. Widened Primitive (long)"); }
    static void print(Integer a)   { System.out.println("2. Autoboxed (Integer)"); }
    static void print(int... args) { System.out.println("3. Varargs (int...)"); }

    public static void main(String[] args) {
        int val = 42;
        print(val); // Compiler prioritizes Widening over Autoboxing!
        // Output: "1. Widened Primitive (long)"
    }
}
```

7.3 Under the Hood of JVM Dynamic Method Dispatch: vtable & itable

How does the JVM know which overridden method to execute when calling `animal.makeSound()` on an `Animal` reference pointing to a `Dog` heap object?

At the bytecode level, the compiler generates the `invokevirtual` instruction. The HotSpot JVM executes dynamic dispatch using the Virtual Method Table (`vtable`):

- **1. Class vtable Creation:** When the ClassLoader loads a class into Metaspace, HotSpot builds an array of method pointers (the `vtable`). The subclass inherits the parent's vtable array slots.
- **2. Method Overriding in vtable:** If the subclass overrides `makeSound()`, the pointer at that specific vtable index is overwritten to point to the subclass's compiled method bytecode.
- **3. Runtime Invocation:** When `invokevirtual` executes: (1) JVM reads the object's Klass Pointer from its header to locate the runtime class metadata, (2) Indexes into the class's `vtable` at the predetermined offset, (3) Jumps directly to the target method bytecode.

Bytecode Instruction	Target Method Type	Dispatch Mechanism
`invokevirtual`	Public and protected non-interface instance methods	Dynamic dispatch via class `vtable`
`invokeinterface`	Methods invoked through an interface reference	Dynamic dispatch via interface table (`itable`)
`invokestatic`	Static methods (`ClassName.method()`)	Static binding at compile time (no `this` pointer)
`invokespecial`	Private methods, constructors (`<init>`), and `super.method()`	Static binding directly to specific class (bypasses dynamic override)
`invokedynamic`	Lambdas and dynamic language invocations (Java 7+)	Bootstrap Method (BSM) linked call site

7.4 Upcasting, Downcasting & Pattern Matching (Java 16+)

Upcasting (assigning a child object to a parent reference) is completely safe and implicit. Downcasting (assigning a parent reference to a child variable) is dangerous and requires an explicit cast. If the underlying object is not an instance of the target type, the JVM throws `ClassCastException`.

Java 16 introduced Pattern Matching for `instanceof`, eliminating tedious boilerplate casting:

```
// --- Modern Java 16+ Pattern Matching for instanceof (JAVA) ---
public class PatternMatchingDemo {
```

```
    // OLD JAVA (Pre-Java 16): Verbose and error-prone
    public static void oldProcess(Object obj) {
        if (obj instanceof String) {
            String s = (String) obj; // Manual, redundant cast!
            System.out.println("Length: " + s.length());
        }
    }

    // MODERN JAVA 16+: Pattern Matching for instanceof
    public static void modernProcess(Object obj) {
        // Declares binding variable 's' directly in the condition!
        if (obj instanceof String s && !s.isBlank()) {
            System.out.println("Clean length: " + s.length()); // 's' is immediately available!
        }
    }
}
```

> **★ Covariant Return Types in Java**
>
> In Java 5+, an overriding method can declare a return type that is a subclass of the superclass method's return type.
>
> Example: If `Animal` declares `public Animal reproduce()`, the `Dog` class may override it as `public Dog reproduce()`.
>
> This eliminates the need for callers to cast the returned object!

Chapter 8: Static Members, Advanced Enums & Nested Classes

Deep exploration of static memory allocation, static method hiding vs. virtual overriding, Java Enums as rich OOP classes, and static vs. inner nested classes.

8.1 Static Variables & Methods in the Metaspace

The `static` keyword binds a member to the Class itself rather than any individual heap instance. Static fields are stored in the Metaspace (or class mirror in the heap) and are initialized when the class is loaded. Static methods do NOT receive an implicit `this` reference; therefore, they cannot access non-static instance fields or call non-static methods directly.

8.2 Static Method Hiding vs. Virtual Overriding (Critical Trap)

A common interview trap: Can static methods be overridden? NO. If a subclass declares a static method with the same signature as a superclass static method, it is called Method Hiding, NOT overriding. Static methods are resolved at compile-time based on the reference type (`invokestatic`), not the runtime object type!

```
// --- Static Method Hiding vs Polymorphic Overriding (JAVA) ---
class Parent {
    public static void display() {
        System.out.println("Parent static display");
    }
}

class Child extends Parent {
    // This HIDES Parent.display(), it does NOT override it!
    public static void display() {
        System.out.println("Child static display");
    }
}

public class StaticHidingDemo {
```

```
    public static void main(String[] args) {
        Parent ref = new Child();
        ref.display();
        // OUTPUT: "Parent static display"!
        // Statically bound to the reference type 'Parent' at compile time!
    }
}
```

8.3 Java Enums as Advanced OOP Classes

Unlike C/C++ where enums are mere integer wrappers, Java Enums (`enum`) are full-blown classes extending `java.lang.Enum`. They can declare private constructors, instance fields, regular methods, abstract methods, and constant-specific class bodies (implementing the Strategy Pattern directly inside the Enum!).

```
// --- Java Enum as a Rich Strategy Pattern Entity (JAVA) ---
public enum Operation {
    PLUS("+") {
        @Override
        public double apply(double x, double y) { return x + y; }
    },
    MINUS("-") {
        @Override
        public double apply(double x, double y) { return x - y; }
    },
    MULTIPLY("*") {
        @Override
        public double apply(double x, double y) { return x * y; }
    },
    DIVIDE("/") {
        @Override
        public double apply(double x, double y) {
            if (y == 0) throw new ArithmeticException("Division by zero");
            return x / y;
        }
    };

    private final String symbol;

    // Enum constructors are strictly private
    Operation(String symbol) {
        this.symbol = symbol;
    }

    public String getSymbol() { return symbol; }

    // Constant-Specific Method: Enforces Strategy Pattern in each constant!
    public abstract double apply(double x, double y);
}

// Clean usage:
// double result = Operation.MULTIPLY.apply(4, 5); // 20.0
```

8.4 Java Nested Classes: Static vs. Inner

Java permits classes to be defined within other classes. Understanding the memory implications of Static Nested Classes vs. Non-Static Inner Classes is vital to avoiding memory leaks:

Category	Declaration	Implicit Outer Reference?	Memory Leak Risk in Async

			/ Listeners
Static Nested Class	`static class Nested {}`	NO (Completely decoupled; cannot access outer instance fields)	Zero leak risk. Ideal for Builders and DTOs.
Non-Static Inner Class	`class Inner {}`	YES (`OuterClass.this` stored in every inner instance)	HIGH: Holds outer instance in memory, preventing GC reclamation.
Anonymous Inner Class	`new Runnable() { ... }`	YES (Holds implicit outer reference if declared in non-static context)	HIGH: Common source of memory leaks in thread pools.
Local Inner Class	Inside a method body	YES (Can also capture `effectively final` local variables)	Moderate: Scoped strictly to method execution.

> **★ Joshua Bloch's Rule on Nested Classes**
>
> Favor static member classes over non-static member classes.
>
> If an inner class does not require access to the enclosing instance's state, ALWAYS mark it `static`. Omitting `static` wastes 4 to 8 bytes per instance for the hidden outer pointer and risks leaking the enclosing object in memory.

Chapter 9: Object Relationships & UML Models in Java

Mastering software coupling taxonomy in Java: Dependency, Association, Aggregation, and Composition, alongside precise UML notation and memory ownership rules.

9.1 The Spectrum of Object Coupling in Java

Modern Java systems design requires precise categorization of how classes interact. Coupling ranges from weak, transient interactions to strong, life-and-death ownership:

Relationship	Semantic Meaning	Ownership Level	Part Lifetime Dependency	UML Representation
Dependency	'Uses-A' (Transient)	None	Transient; passed as method parameter or local variable	Dashed line with arrow: `- - - >`
Association	'Knows-A' / 'Peer'	Peer link	Independent; both objects exist separately in the heap	Solid line: `————————`
Aggregation	'Has-A' (Weak Whole-Part)	Shared / Non-owning	Independent; Part survives if Whole is garbage collected	Hollow diamond at Whole: `◇————`
Composition	'Has-A' (Strong Whole-Part)	Exclusive / Owning	Bound; Part is created inside Whole and collected with it	Filled solid diamond at Whole: `◆————`
Generalization	'Is-A' (Inheritance)	Taxonomic	Subclass `extends` Superclass	Solid line with hollow triangle: `———▷`
Realization	'Implements' (Contract)	Contractual	Class `implements` Interface	Dashed line with hollow triangle: `- - ▷`

9.2 Java Code Dissection: Aggregation vs. Composition

A critical interview question: How do you implement Aggregation versus Composition in Java, where all objects live on the heap and there are no C++ value members?

In Java, the distinction is enforced through LIFETIME RESPONSIBILITY and CONSTRUCTOR INJECTION:

```
// --- Java Implementation of Dependency, Aggregation, and Composition (JAVA) ---
import java.util.List;
import java.util.ArrayList;

// 1. DEPENDENCY ("Uses-A"): Service is passed temporarily to execute()
class EmailService {
    public void sendEmail(String to, String msg) { System.out.println("Email sent to " + to); }
}

class UserRegistration {
    public void register(String email, EmailService emailer) { // Transient dependency!
        emailer.sendEmail(email, "Welcome to the Platform!");
    }
}

// 2. AGGREGATION ("Has-A" Weak): Department aggregates Professors
// If the Department is garbage collected, the Professors STILL exist!
class Professor {
    private final String name;
    public Professor(String name) { this.name = name; }
}

class Department {
    private final String deptName;
    private final List<Professor> faculty; // Aggregated reference list

    // Injected from the outside (Aggregation)
    public Department(String name, List<Professor> faculty) {
        this.deptName = name;
        this.faculty = new ArrayList<>(faculty);
    }
}

// 3. COMPOSITION ("Has-A" Strong): Order is composed of OrderItems
// If the Order is deleted, its OrderItems have NO architectural meaning and are destroyed!
class OrderItem {
    private final String sku;
    private final int quantity;
    public OrderItem(String sku, int qty) { this.sku = sku; this.quantity = qty; }
}

class Order {
    private final String orderId;
    private final List<OrderItem> items; // Composition: Owned exclusively!

    public Order(String orderId) {
        this.orderId = orderId;
        this.items = new ArrayList<>(); // Items created and managed strictly inside Order
    }

    public void addItem(String sku, int qty) {
        this.items.add(new OrderItem(sku, qty)); // Exclusive lifecycle ownership
    }
}
```

> **★ How to Decide: Aggregation vs. Composition in Java**

> Ask yourself: 'Can the child object exist without the parent object?'
> 1. If YES -> Use AGGREGATION (Inject the dependency into the parent class from the outside).
> 2. If NO -> Use COMPOSITION (Instantiate and manage the child object exclusively within the parent, and ensure cascading deletion).

Chapter 10: The SOLID Design Principles (Java Edition)

Exhaustive architectural breakdown of all 5 SOLID principles with real-world violating anti-patterns and clean, production-grade Java refactored code.

10.1 Single Responsibility Principle (SRP)

Definition (Robert C. Martin): 'A class should have one, and only one, reason to change.' A class should be responsible to only one actor or business domain. When a class mixes business calculations, persistence, and reporting, changes to database schemas or HTML formats threaten the stability of the core business logic.

```
// --- SRP Violation vs Clean Refactored Java Architecture (JAVA) ---
// VIOLATING SRP: God Object with 3 distinct reasons to change!
public class Invoice {
    private double amount;
    public Invoice(double amt) { this.amount = amt; }

    // Reason 1: Business Calculation
    public double calculateTotalWithTax() { return amount * 1.15; }

    // Reason 2: Database Persistence (Violates SRP!)
    public void saveToDatabase() {
        System.out.println("Executing SQL: INSERT INTO invoices VALUES (" + amount + ")");
    }

    // Reason 3: Presentation / Formatting (Violates SRP!)
    public void printInvoiceHTML() {
        System.out.println("<html><body>Total: " + calculateTotalWithTax() + "</body></html>");
    }
}

// ------------------------------------------------------------
// REFACTORED SRP: Cohesive, Segregated Classes
// ------------------------------------------------------------
public class Invoice {
    private final double amount;
    public Invoice(double amt) { this.amount = amt; }
    public double getAmount() { return amount; }
    public double calculateTotal(double taxRate) { return amount * (1.0 + taxRate); }
}

public class InvoiceRepository {
    public void save(Invoice invoice) {
        System.out.println("Saving invoice to DB: " + invoice.getAmount());
    }
}

public class InvoiceHtmlPrinter {
    public void print(Invoice invoice) {
        System.out.println("<html><body>Total: " + invoice.calculateTotal(0.15) + "</body></html>");
    }
```

```
}
```

10.2 Open/Closed Principle (OCP)

Definition: 'Software entities should be open for extension, but closed for modification.' You should be able to introduce new functionality without altering existing, tested source code. Achieved primarily via polymorphic interfaces and strategy patterns.

```
// --- OCP Violation vs Polymorphic Open/Closed Design (JAVA) ---
// VIOLATING OCP: Every new payment method requires editing this class!
public class FaultyPaymentProcessor {
    public void processPayment(String paymentType, double amount) {
        if ("CREDIT_CARD".equalsIgnoreCase(paymentType)) {
            System.out.println("Charging Credit Card: $" + amount);
        } else if ("PAYPAL".equalsIgnoreCase(paymentType)) {
            System.out.println("Routing to PayPal: $" + amount);
        } else if ("BITCOIN".equalsIgnoreCase(paymentType)) { // Forces code modification!
            System.out.println("Transferring Bitcoin: $" + amount);
        }
    }
}

// ------------------------------------------------------------
// REFACTORED OCP: Polymorphic Open/Closed Architecture
// ------------------------------------------------------------
public interface PaymentMethod {
    void pay(double amount);
}

public class CreditCardPayment implements PaymentMethod {
    public void pay(double amount) { System.out.println("Charging Credit Card: $" + amount); }
}

public class PayPalPayment implements PaymentMethod {
    public void pay(double amount) { System.out.println("Routing to PayPal: $" + amount); }
}

// PaymentService is CLOSED to modification, but OPEN to new payment extensions!
public class PaymentService {
    public void executePayment(PaymentMethod method, double amount) {
        method.pay(amount); // Polymorphic dispatch!
    }
}
```

10.3 Liskov Substitution Principle (LSP)

Definition (Barbara Liskov, 1987): 'Subtypes must be substitutable for their base types without altering the correctness of the program.' If a client program uses a base class reference, passing any subclass must never produce unexpected side effects, broken invariants, or unnatural exceptions.

The Classic LSP Trap: A Square is a Rectangle mathematically, but NOT in object-oriented behavior. A Rectangle allows width and height to vary independently; forcing a Square into this hierarchy forces setters to mutate both dimensions simultaneously, breaking caller expectations.

```
// --- The Classic Rectangle-Square LSP Violation (JAVA) ---
// VIOLATING LSP: Square mutates expected Rectangle behavior
public class Rectangle {
    protected int width, height;
```

```
    public void setWidth(int w) { this.width = w; }
    public void setHeight(int h) { this.height = h; }
    public int getArea() { return width * height; }
}

public class Square extends Rectangle {
    @Override
    public void setWidth(int w) { this.width = this.height = w; }  // Unexpected side effect!
    @Override
    public void setHeight(int h) { this.width = this.height = h; } // Unexpected side effect!
}

public class LSPTester {
    public static void verify(Rectangle r) {
        r.setWidth(10);
        r.setHeight(5);
        // Expectation: Area MUST be 10 * 5 = 50!
        // But if 'r' is a Square, area is 5 * 5 = 25! LSP VIOLATION!
        assert r.getArea() == 50 : "Broken assumption!";
    }
}
```

10.4 Interface Segregation Principle (ISP)

Definition: 'Clients should not be forced to depend upon interfaces that they do not use.' Fat, bloated interfaces force implementers to write empty method stubs or throw `UnsupportedOperationException`. Break monolithic interfaces into small, cohesive, role-based contracts.

```
// --- ISP Fat Interface vs Granular Segregated Interfaces (JAVA) ---
// VIOLATING ISP: Fat Interface forcing unnatural dummy implementations
public interface Worker {
    void work();
    void eat();
    void sleep();
}

public class RobotWorker implements Worker {
    public void work() { System.out.println("Robot working 24/7"); }
    public void eat() { throw new UnsupportedOperationException("Robots do not eat!"); } // POLLUTION!
    public void sleep() { throw new UnsupportedOperationException("Robots do not sleep!"); } //
POLLUTION!
}

// ------------------------------------------------------------
// REFACTORED ISP: Segregated Role Interfaces
// ------------------------------------------------------------
public interface Workable { void work(); }
public interface Feedable { void eat(); }
public interface Sleepable { void sleep(); }

public class HumanWorker implements Workable, Feedable, Sleepable {
    public void work() { System.out.println("Human working"); }
    public void eat() { System.out.println("Human eating lunch"); }
    public void sleep() { System.out.println("Human sleeping"); }
}

public class Robot implements Workable {
    public void work() { System.out.println("Robot assembling electronics"); } // Zero bloat!
}
```

10.5 Dependency Inversion Principle (DIP)

Definition: 'High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.' High-level business orchestrators should never instantiate low-level databases or notification clients directly. Instead, dependencies are injected via interfaces (Inversion of Control via Constructor Injection).

```
// --- Dependency Inversion via Constructor Injection in Java (JAVA) ---
// REFACTORED DIP: High-level business logic decoupled from low-level details
public interface NotificationChannel {
    void sendNotification(String recipient, String message);
}

public class EmailNotificationChannel implements NotificationChannel {
    public void sendNotification(String recipient, String message) {
        System.out.println("Sending SMTP email to " + recipient + ": " + message);
    }
}

public class SmsNotificationChannel implements NotificationChannel {
    public void sendNotification(String recipient, String message) {
        System.out.println("Sending SMS text to " + recipient + ": " + message);
    }
}

// High-level service depends strictly on the NotificationChannel abstraction:
public class OrderNotificationService {
    private final NotificationChannel channel; // Injected abstraction

    // Constructor Injection (Dependency Inversion / IoC)
    public OrderNotificationService(NotificationChannel channel) {
        this.channel = channel;
    }

    public void notifyCustomer(String customer, String orderStatus) {
        channel.sendNotification(customer, "Order update: " + orderStatus);
    }
}
```

★ The SOLID Health Checklist for Senior Engineers

S: Does your class have more than one reason to change? If so, break it into cohesive services.

O: Can you add new business capabilities without editing previously tested `.java` files?

L: Can any child class replace its parent without throwing unexpected runtime exceptions?

I: Are your interfaces clean and role-specific, or do they force classes to implement empty dummy methods?

D: Are your services accepting interfaces via constructor injection rather than using `new ConcreteService()`?

Chapter 11: Essential Gang of Four (GoF) Design Patterns in Java

Mastering the 8 most critical industry design patterns across Creational, Structural, and Behavioral categories with thread-safe, production-ready Java code.

11.1 The Gang of Four Taxonomy in Enterprise Java

Design patterns represent reusable, battle-tested solutions to recurring software engineering challenges. Categorized by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (GoF), they span three operational domains:

- **1. Creational Patterns:** Isolate instantiation logic, ensuring the client remains decoupled from concrete classes.
- **2. Structural Patterns:** Compose classes and objects into flexible, larger architectures without tight coupling.
- **3. Behavioral Patterns:** Orchestrate algorithms, state transitions, and communication contracts between objects.

11.2 Creational: The Three Thread-Safe Java Singletons

A Singleton ensures only one instance exists and provides a global access point. In multithreaded enterprise environments, naive singletons create duplicate instances due to thread race conditions.

```
// --- The Three Thread-Safe Java Singleton Implementations (JAVA) ---
// 1. Double-Checked Locking Singleton (Industry Standard)
public class DclSingleton {
    // 'volatile' is MANDATORY to prevent CPU instruction reordering!
    // (1. Allocate memory -> 2. Initialize instance -> 3. Point reference)
    private static volatile DclSingleton instance;

    private DclSingleton() {}

    public static DclSingleton getInstance() {
        if (instance == null) { // 1st check: Avoids lock overhead once initialized
            synchronized (DclSingleton.class) {
                if (instance == null) { // 2nd check: Inside critical section
                    instance = new DclSingleton();
                }
            }
        }
        return instance;
    }
}

// 2. Bill Pugh Static Inner Class Singleton (Lazy, Zero-Lock Overhead)
public class BillPughSingleton {
    private BillPughSingleton() {}

    // Inner class is loaded into memory ONLY when getInstance() is called!
    private static class Holder {
        private static final BillPughSingleton INSTANCE = new BillPughSingleton();
    }

    public static BillPughSingleton getInstance() {
        return Holder.INSTANCE;
    }
}

// 3. Joshua Bloch's Enum Singleton (100% Reflection & Serialization Proof)
public enum EnumSingleton {
    INSTANCE;
    public void executeBusinessLogic() {
        System.out.println("Executing thread-safe operation");
    }
}
```

11.3 Creational: Fluent Builder Pattern

The Builder pattern solves the 'Telescoping Constructor' anti-pattern where constructors accept 10+ confusing parameters with multiple booleans and defaults.

```
// --- Java Fluent Builder Pattern (JAVA) ---
public class HttpRequest {
    private final String url;
    private final String method;
    private final String body;
    private final int timeoutMs;

    private HttpRequest(Builder builder) {
        this.url = builder.url;
        this.method = builder.method;
        this.body = builder.body;
        this.timeoutMs = builder.timeoutMs;
    }

    public static class Builder {
        private final String url;          // Required parameter
        private String method = "GET";     // Optional with default
        private String body = "";
        private int timeoutMs = 5000;

        public Builder(String url) { this.url = url; }
        public Builder method(String method) { this.method = method; return this; }
        public Builder body(String body) { this.body = body; return this; }
        public Builder timeout(int ms) { this.timeoutMs = ms; return this; }

        public HttpRequest build() { return new HttpRequest(this); }
    }
}

// Fluent Chaining Usage:
HttpRequest req = new HttpRequest.Builder("https://api.payments.com/charge")
        .method("POST")
        .body("{\"amount\": 100.0}")
        .timeout(2500)
        .build();
```

11.4 Structural: Decorator Pattern (Dynamic Behavior Stacking)

The Decorator Pattern attaches additional responsibilities to an object dynamically at runtime without subclassing (the exact architecture used by `java.io.BufferedReader(new InputStreamReader(...))`):

```
// --- Java Decorator Pattern Dynamic Stacking (JAVA) ---
public interface Coffee {
    String getDescription();
    double getCost();
}

public class Espresso implements Coffee {
    public String getDescription() { return "Espresso"; }
    public double getCost() { return 2.50; }
}

// Base Decorator wraps Coffee
public abstract class CoffeeDecorator implements Coffee {
    protected final Coffee decoratedCoffee;
    public CoffeeDecorator(Coffee coffee) { this.decoratedCoffee = coffee; }
}
```

```
public class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) { super(coffee); }
    public String getDescription() { return decoratedCoffee.getDescription() + ", Steamed Milk"; }
    public double getCost() { return decoratedCoffee.getCost() + 0.60; }
}

public class CaramelDecorator extends CoffeeDecorator {
    public CaramelDecorator(Coffee coffee) { super(coffee); }
    public String getDescription() { return decoratedCoffee.getDescription() + ", Caramel Drizzle"; }
    public double getCost() { return decoratedCoffee.getCost() + 0.75; }
}

// Dynamic Stacking:
// Coffee myOrder = new CaramelDecorator(new MilkDecorator(new Espresso()));
// Cost: 2.50 + 0.60 + 0.75 = $3.85
```

11.5 Behavioral: Observer & Strategy Patterns

The Observer Pattern implements a one-to-many publish-subscribe model. The Strategy Pattern encapsulates interchangeable algorithms behind a shared interface.

```
// --- Java Strategy Pattern Interchangeable Algorithms (JAVA) ---
// Java Strategy Pattern: Dynamic Sorting Algorithms
import java.util.List;

public interface SortStrategy {
    void sort(List<Integer> dataset);
}

public class QuickSortStrategy implements SortStrategy {
    public void sort(List<Integer> dataset) { System.out.println("Sorting via O(N log N)
QuickSort"); }
}

public class MergeSortStrategy implements SortStrategy {
    public void sort(List<Integer> dataset) { System.out.println("Sorting via Stable MergeSort"); }
}

public class SortContext {
    private SortStrategy strategy;
    public void setStrategy(SortStrategy strategy) { this.strategy = strategy; }
    public void executeSort(List<Integer> data) {
        if (strategy == null) throw new IllegalStateException("Strategy not set");
        strategy.sort(data); // Dynamic polymorphic execution!
    }
}
```

★ Gang of Four Pattern Selection Matrix

Need only one global instance with safe initialization? -> Singleton (Double-Checked or Enum)

Complex constructor with multiple optional parameters? -> Builder

Decouple object instantiation from calling code? -> Factory Method

Bridge incompatible third-party interfaces? -> Adapter

Dynamically stack features at runtime without subclass explosion? -> Decorator

Interchangeable business algorithms or payment processors? -> Strategy

Event-driven pub/sub state broadcasting? -> Observer

Chapter 12: Production Low-Level Design (LLD): Parking Lot System in Java

A complete, runnable, production-grade Low-Level Design (LLD) case study of a multi-floor Parking Lot System applying SOLID, Strategy, Singleton, and thread-safe spot reservation.

12.1 System Requirements & Functional Specifications

Low-Level Design (LLD) machine-coding interviews at companies like Amazon, Google, Uber, and Microsoft evaluate your ability to write modular, extensible, and thread-safe object-oriented code. The specifications for this Parking Lot System include:

- **1. Multi-Floor Architecture:** The parking lot spans multiple levels, each housing spots tailored to specific vehicle sizes.
- **2. Diverse Vehicle Dimensions:** Supports Motorcycles, Compact Cars, and Large Trucks. Spots can only accommodate compatible or smaller vehicle types.
- **3. Dynamic Pricing Strategy:** Parking fees are computed dynamically via interchangeable algorithms (Strategy Pattern).
- **4. Thread Safety:** Concurrent entry gates must safely assign spots without race conditions (synchronized spot locking).

12.2 Production-Grade Java Implementation

The following clean, modular Java code implements the entire domain model and operational workflows:

```
// --- Complete Production LLD: Java Parking Lot System (JAVA) ---
import java.time.LocalDateTime;
import java.time.Duration;
import java.util.List;
import java.util.ArrayList;
import java.util.UUID;
import java.util.Optional;

// 1. Vehicle Hierarchy & Enums
public enum VehicleType { MOTORCYCLE, CAR, TRUCK }

public abstract class Vehicle {
    private final String licensePlate;
    private final VehicleType type;

    public Vehicle(String licensePlate, VehicleType type) {
        this.licensePlate = licensePlate;
        this.type = type;
    }

    public String getLicensePlate() { return licensePlate; }
    public VehicleType getType() { return type; }
}

public class Car extends Vehicle {
    public Car(String plate) { super(plate, VehicleType::CAR == null ? VehicleType.CAR :
VehicleType.CAR); }
}

public class Motorcycle extends Vehicle {
```

```
    public Motorcycle(String plate) { super(plate, VehicleType.MOTORCYCLE); }
}

public class Truck extends Vehicle {
    public Truck(String plate) { super(plate, VehicleType.TRUCK); }
}

// 2. Parking Spot Entity
public class ParkingSpot {
    private final int spotId;
    private final VehicleType supportedType;
    private boolean isAvailable;
    private Vehicle parkedVehicle;

    public ParkingSpot(int id, VehicleType type) {
        this.spotId = id;
        this.supportedType = type;
        this.isAvailable = true;
    }

    public synchronized boolean canFitVehicle(Vehicle vehicle) {
        return isAvailable && (vehicle.getType().ordinal() <= supportedType.ordinal());
    }

    public synchronized void assignVehicle(Vehicle vehicle) {
        this.parkedVehicle = vehicle;
        this.isAvailable = false;
    }

    public synchronized void vacate() {
        this.parkedVehicle = null;
        this.isAvailable = true;
    }

    public int getSpotId() { return spotId; }
    public boolean isAvailable() { return isAvailable; }
    public VehicleType getSupportedType() { return supportedType; }
}

// 3. Strategy Pattern: Fee Calculation
public interface PricingStrategy {
    double calculateFee(long durationHours, VehicleType type);
}

public class HourlyPricingStrategy implements PricingStrategy {
    @Override
    public double calculateFee(long durationHours, VehicleType type) {
        long billableHours = Math.max(1, durationHours);
        return switch (type) {
            case MOTORCYCLE -> billableHours * 2.0;
            case CAR -> billableHours * 5.0;
            case TRUCK -> billableHours * 10.0;
        };
    }
}

// 4. Ticket Value Object
public class ParkingTicket {
    private final String ticketId;
    private final Vehicle vehicle;
    private final int spotId;
    private final int floorId;
    private final LocalDateTime entryTime;
```

```
    public ParkingTicket(Vehicle vehicle, int spotId, int floorId) {
        this.ticketId = "TKT-" + UUID.randomUUID().toString().substring(0, 8);
        this.vehicle = vehicle;
        this.spotId = spotId;
        this.floorId = floorId;
        this.entryTime = LocalDateTime.now();
    }

    public String getTicketId() { return ticketId; }
    public Vehicle getVehicle() { return vehicle; }
    public int getSpotId() { return spotId; }
    public int getFloorId() { return floorId; }
    public LocalDateTime getEntryTime() { return entryTime; }
}

// 5. Parking Floor (Aggregates Spots)
public class ParkingFloor {
    private final int floorId;
    private final List<ParkingSpot> spots;

    public ParkingFloor(int floorId) {
        this.floorId = floorId;
        this.spots = new ArrayList<>();
    }

    public void addSpot(ParkingSpot spot) { spots.add(spot); }

    public synchronized Optional<ParkingSpot> findAvailableSpot(Vehicle vehicle) {
        return spots.stream()
                .filter(spot -> spot.canFitVehicle(vehicle))
                .findFirst();
    }

    public int getFloorId() { return floorId; }
    public List<ParkingSpot> getSpots() { return spots; }
}

// 6. ParkingLot Controller (Thread-Safe Singleton Facade)
public class ParkingLot {
    private static volatile ParkingLot instance;
    private final List<ParkingFloor> floors;
    private PricingStrategy pricingStrategy;

    private ParkingLot() {
        this.floors = new ArrayList<>();
        this.pricingStrategy = new HourlyPricingStrategy(); // Default strategy
    }

    public static ParkingLot getInstance() {
        if (instance == null) {
            synchronized (ParkingLot.class) {
                if (instance == null) {
                    instance = new ParkingLot();
                }
            }
        }
        return instance;
    }

    public void addFloor(ParkingFloor floor) { floors.add(floor); }
    public void setPricingStrategy(PricingStrategy strategy) { this.pricingStrategy = strategy; }

    public synchronized ParkingTicket parkVehicle(Vehicle vehicle) {
        for (ParkingFloor floor : floors) {
```

```
            Optional<ParkingSpot> spotOpt = floor.findAvailableSpot(vehicle);
            if (spotOpt.isPresent()) {
                ParkingSpot spot = spotOpt.get();
                spot.assignVehicle(vehicle);
                return new ParkingTicket(vehicle, spot.getSpotId(), floor.getFloorId());
            }
        }
        throw new IllegalStateException("Parking Full: No compatible spot found for " +
vehicle.getType());
    }

    public synchronized double exitVehicle(ParkingTicket ticket, long simulatedHours) {
        double fee = pricingStrategy.calculateFee(simulatedHours, ticket.getVehicle().getType());
        // Vacate spot
        for (ParkingFloor floor : floors) {
            if (floor.getFloorId() == ticket.getFloorId()) {
                floor.getSpots().stream()
                        .filter(s -> s.getSpotId() == ticket.getSpotId())
                        .findFirst()
                        .ifPresent(ParkingSpot::vacate);
            }
        }
        System.out.println("Ticket " + ticket.getTicketId() + " processed. Total Due: $" + fee);
        return fee;
    }
}
```

★ Design Review & Applied Principles

1. Single Responsibility (SRP): ParkingSpot encapsulates physical spot occupancy; ParkingFloor indexes floors; ParkingLot coordinates ticketing and billing.

2. Open/Closed (OCP): New fee structures (e.g., SurgePricingStrategy) plug directly into the PricingStrategy interface with zero changes to existing code.

3. Concurrency Safety: Critical state transitions in ParkingSpot and ParkingLot use Java `synchronized` monitors to eliminate race conditions during high-volume entries.

Chapter 13: Top 35+ Java OOP Technical Interview Questions & High-Scoring Model Answers

Curated, definitive answers to the most frequently asked Java OOP and JVM questions in FAANG, top-tier engineering interviews, and enterprise systems design exams.

Q1: Is Java strictly pass-by-value or pass-by-reference?

High-Scoring Model Answer: Java is strictly 100% PASS-BY-VALUE. When passing primitive variables, a copy of the raw bits is passed. When passing object references, a copy of the 32-bit or 64-bit reference address pointing to the heap is passed by value. Modifying an object's fields mutates the shared heap object, but reassigning the reference variable itself (`obj = new Object()`) changes only the callee's local stack copy, leaving the caller's reference unchanged.

Q2: What is the exact technical difference between Abstraction and Encapsulation?

High-Scoring Model Answer: Abstraction is OUTSIDE-IN interface design: hiding internal implementation complexity and exposing only the essential public contract to clients (e.g., `List` interface hides array/linked-node

mechanics). Encapsulation is INSIDE-OUT boundary protection: bundling fields with methods and restricting direct field access via `private` access modifiers to preserve class invariants.

Q3: Why doesn't Java support multiple class inheritance?

High-Scoring Model Answer: To eliminate the Diamond Problem and state ambiguity. If Class A defines instance field `int x`, and classes B and C inherit from A, and D inherits from both B and C: (1) Does D have one copy or two copies of `x` in its heap layout? (2) Which constructor initializes `x`? Java avoids this memory layout bloat by allowing single class inheritance (`extends`), but permitting multiple interface implementation (`implements`).

Q4: What is the contract between equals() and hashCode() in Java?

High-Scoring Model Answer: 1. If `o1.equals(o2)` is true, their `hashCode()` values MUST be identical. 2. If `o1.hashCode() == o2.hashCode()`, the objects are NOT necessarily equal (hash collision). Violating this contract breaks `HashMap` and `HashSet`: equal objects are placed into different hash buckets, making them impossible to retrieve via `.get()`.

Q5: Why is String immutable in Java?

High-Scoring Model Answer: 1. Security: Strings are used for database URLs, file paths, and network sockets; immutability prevents unauthorized tampering. 2. Thread Safety: Safe across concurrent threads without locks. 3. String Constant Pool (SCP): Allows the JVM to cache identical string literals, saving massive heap RAM. 4. HashCode Caching: The hash code is computed once during creation and cached (`private int hash`), making `HashMap` lookups O(1).

Q6: What is the difference between String, StringBuilder, and StringBuffer?

High-Scoring Model Answer: `String` is immutable; every modification creates a new heap object. `StringBuilder` is mutable and not thread-safe (fastest for single-threaded string concatenation). `StringBuffer` is mutable and thread-safe (its methods are `synchronized`, incurring lock overhead).

Q7: What is the Java Integer Cache and what is its range?

High-Scoring Model Answer: To save heap memory, the Java compiler caches `Integer` objects for values from -128 to 127 via `Integer.valueOf()`. Comparing two `Integer` references in this range using `==` returns `true` (same cached instance), but comparing values >= 128 returns `false` (different heap instances). Always compare wrappers using `.equals()`!

Q8: Can static methods be overridden in Java?

High-Scoring Model Answer: NO. If a subclass declares a static method with the same signature, it is Method Hiding, not overriding. Static methods are bound at compile time based on the reference type via the `invokestatic` bytecode instruction, bypassing the virtual method table (`vtable`).

Q9: Can a constructor be declared final, static, or abstract?

High-Scoring Model Answer: No. A constructor cannot be `final` because constructors are never inherited and cannot be overridden. A constructor cannot be `static` because it belongs strictly to instance initialization and requires a `this` reference. A constructor cannot be `abstract` because it must instantiate the object's memory directly.

Q10: Why was finalize() deprecated and marked for removal?

High-Scoring Model Answer: 1. Non-deterministic execution: The JVM provides no guarantee when or if finalizers run. 2. Object resurrection: Finalizers can reassign `this` to a static field, resurrecting dead objects. 3.

Performance penalty: Finalizable objects must survive at least one extra GC cycle. Solution: Use `AutoCloseable` with `try-with-resources`.

Q11: How does try-with-resources work and what interface is required?

High-Scoring Model Answer: Introduced in Java 7, `try-with-resources` automatically closes any resource implementing `java.lang.AutoCloseable` when the `try` block exits. It guarantees deterministic cleanup even if exceptions are thrown and properly captures suppressed exceptions via `Throwable.getSuppressed()`.

Q12: Explain the Weak Generational Hypothesis in JVM Garbage Collection.

High-Scoring Model Answer: Empirical evidence proves >95% of allocated objects die shortly after creation. HotSpot divides the Heap into Young Generation (Eden, S0, S1) and Old Generation. Young Gen uses fast Minor GC copying algorithms. Objects surviving 15 GC cycles are promoted to Old Gen, which is collected less frequently via Major/Full GC.

Q13: What are the 4 Reference Types in Java (java.lang.ref)?

High-Scoring Model Answer: 1. StrongReference: Standard reference; never collected while reachable. 2. SoftReference: Collected only before OutOfMemoryError; used for memory-sensitive caches. 3. WeakReference: Collected on the very next GC run; used in `WeakHashMap` to prevent memory leaks. 4. PhantomReference: Enqueued after finalization; used for low-level off-heap resource cleanup.

Q14: How do vtable and itable work under the hood during dynamic method dispatch?

High-Scoring Model Answer: For each loaded class, the HotSpot JVM constructs a Virtual Method Table (`vtable`) in Metaspace containing an array of method pointers. When `invokevirtual` executes: (1) Reads Klass pointer from object header, (2) Indexes into the class's `vtable` at a fixed offset, (3) Jumps directly to the overridden method bytecode. Interface references use an `itable` (Interface Table).

Q15: What are the differences between invokevirtual, invokespecial, and invokestatic?

High-Scoring Model Answer: `invokevirtual` performs dynamic dispatch via `vtable` for public/protected instance methods. `invokestatic` performs static binding for static methods. `invokespecial` performs static binding for constructors (`<init>`), private methods, and `super.method()` calls.

Q16: What are Covariant Return Types in Java?

High-Scoring Model Answer: An overriding subclass method may declare a return type that is a subtype of the superclass method's return type (e.g., `public Dog clone()` overriding `public Animal clone()`). This eliminates the need for manual downcasting by the caller.

Q17: What are the 4 strict rules of Method Overriding in Java?

High-Scoring Model Answer: 1. Signature (name and parameter types) must match exactly. 2. Access modifier cannot be more restrictive than the parent method. 3. Return type must be identical or covariant. 4. The overriding method CANNOT throw new or broader CHECKED exceptions.

Q18: What is the difference between Checked and Unchecked Exceptions in Java OOP?

High-Scoring Model Answer: Checked Exceptions (subclasses of `Exception` excluding `RuntimeException`) represent recoverable external failures (e.g., `IOException`, `SQLException`) and are verified at compile time. Unchecked Exceptions (subclasses of `RuntimeException` and `Error`) represent programming bugs (e.g., `NullPointerException`, `IndexOutOfBoundsException`) and do not require explicit handling.

Q19: What is the Liskov Substitution Principle (LSP) and what is the classic violation?

High-Scoring Model Answer: Subtypes must be substitutable for their base types without altering program correctness. The classic violation is `Square extends Rectangle`: mutating width inadvertently mutates height, breaking tests that assume width and height are independent dimensions.

Q20: Why should we favor Composition over Inheritance (Fragile Base Class)?

High-Scoring Model Answer: Inheritance creates tight white-box coupling where changes in superclass method dispatch (e.g., `HashSet.addAll()` calling `add()`) break subclass counting invariants. Composition establishes loose black-box coupling through interfaces, allowing behaviors to be mocked in unit tests and swapped at runtime.

Q21: Why is volatile mandatory in Double-Checked Locking Singletons?

High-Scoring Model Answer: Object creation `instance = new Singleton()` involves 3 JVM steps: (1) Allocate memory, (2) Initialize object fields, (3) Assign address to `instance`. Without `volatile`, the CPU/JIT compiler may reorder steps to 1 -> 3 -> 2. Another thread checking `instance == null` sees a non-null reference to a partially-initialized object, causing catastrophic runtime crashes.

Q22: Why does Joshua Bloch recommend Enum Singletons?

High-Scoring Model Answer: An Enum Singleton (`public enum Singleton { INSTANCE; }`) is 100% thread-safe by the JVM classloader, immune to reflection attacks (calling `Constructor.newInstance()` on enums throws `IllegalArgumentException`), and handles serialization automatically without duplicate instances.

Q23: What is the difference between Static Nested Classes and Non-Static Inner Classes?

High-Scoring Model Answer: A static nested class (`static class Nested {}`) is completely decoupled and does NOT hold a reference to the outer instance. A non-static inner class (`class Inner {}`) contains an implicit hidden reference `OuterClass.this`. If an inner instance is passed to a background thread or static collection, it prevents the outer instance from being garbage collected, causing a memory leak.

Q24: What is Defensive Copying?

High-Scoring Model Answer: Creating clones of mutable objects during constructor assignment and getter retrieval. If a class stores a `Date` or `List` directly, external callers can mutate private state through their retained reference. Defensive copying preserves strict encapsulation.

Q25: What are the 5 rules for creating an Immutable Class in Java?

High-Scoring Model Answer: 1. No setter methods. 2. Mark class `final` to prevent subclassing. 3. Make all fields `final`. 4. Make all fields `private`. 5. Use defensive copying for any mutable field.

Q26: What are Java Records (Java 16+)?

High-Scoring Model Answer: Records (`record Point(int x, int y) {}`) are transparent, immutable data carriers. The compiler automatically generates `final` fields, canonical constructor, accessors (`x()` and `y()`), `equals()`, `hashCode()`, and `toString()`. Records cannot extend other classes (they extend `java.lang.Record`) and are implicitly `final`.

Q27: What is the 'Tell, Don't Ask' principle?

High-Scoring Model Answer: Command objects to execute operations on their own data instead of asking for their data, performing calculations externally, and setting the result back. It preserves domain logic encapsulation within the entity.

Q28: What is an Anemic Domain Model?

High-Scoring Model Answer: An entity that contains only fields and mindless getters and setters without business validation. Business logic leaks into service layers, leading to code duplication and broken domain invariants.

Q29: What is the Lapsed Listener Problem in Java?

High-Scoring Model Answer: When an observer registers with a subject, the subject holds a strong reference to it. If the client discards the observer without unsubscribing, the Garbage Collector cannot reclaim it, causing a permanent memory leak. Solution: Use `WeakReference` or explicit lifecycle deregistration.

Q30: Why were default methods added to Java 8 interfaces?

High-Scoring Model Answer: To allow interface evolution without breaking backwards compatibility. For instance, Java 8 added `.stream()` to `java.util.Collection`; default methods allowed existing collection implementations across the globe to compile and run without needing to implement `.stream()` manually.

Q31: How does Java resolve Interface Default Method collisions (Diamond Problem)?

High-Scoring Model Answer: If a class implements two interfaces sharing an identical default method signature, the compiler throws an ambiguity error. The implementing class MUST override the method and explicitly invoke the desired interface implementation using `InterfaceName.super.method()`.

Q32: What is the Dependency Inversion Principle (DIP)?

High-Scoring Model Answer: High-level business policies should not depend on low-level implementation details; both must depend on abstractions. Implemented via Constructor Dependency Injection, allowing databases or network clients to be swapped without modifying business services.

Q33: What is Pattern Matching for instanceof (Java 16)?

High-Scoring Model Answer: A modern Java syntax enhancement that combines type checking and casting into a single atomic expression: `if (obj instanceof String s) { System.out.println(s.length()); }`, eliminating redundant, unsafe casts.

Q34: What is the difference between Factory Method and Abstract Factory?

High-Scoring Model Answer: Factory Method relies on inheritance to instantiate a single product via an overridden method. Abstract Factory uses object composition to produce complete families of related or dependent objects without specifying their concrete classes.

Q35: Can an interface have private methods in Java?

High-Scoring Model Answer: Yes! Starting in Java 9, interfaces can declare `private` and `private static` methods. They are used to share common helper logic between multiple `default` methods without exposing the code to the public API.

> **★ FAANG Technical Interview Strategy**
>
> 1. Ground every Java OOP answer in JVM internals: mention memory layout, stack vs heap, classloader, or HotSpot vtable dispatch.
> 2. When discussing design principles, connect high-level rules (SOLID, GoF) to concrete production trade-offs: latency, thread safety, and maintainability.
> 3. Always highlight modern Java features (Records, Pattern Matching, try-with-resources) to demonstrate current enterprise-level competence.